

- SGOOLI PARTNER DASHBOARD
 - Project Appendix: Technical Documentation & Evidence
 - APPENDIX A: APPLICATION ARCHITECTURE DIAGRAM
 - System Component Topology
 - APPENDIX B: FOLDER & MODULE STRUCTURE
 - Complete Directory Tree (Analysis-Based)
 - APPENDIX C: KEY TYPESCRIPT CODE SNIPPETS WITH ANNOTATIONS
 - 1. Authentication Flow - useAuth Hook
 - 2. Partner Access Control Hook
 - 3. Permission Provider with Partner-Type Enforcement
 - 4. Wallet Domain Logic - Pure Business Rules
 - 5. Wallet Infrastructure Service
 - 6. Generic CRUD Helpers
 - 7. Campaign Section with Permission Guards
 - APPENDIX D: CONFIGURATION FILES ANALYSIS
 - 1. package.json - Dependency & Script Configuration
 - 2. vite.config.ts - Build Configuration
 - 3. tsconfig.app.json - TypeScript Compiler Options
 - 4. eslint.config.js - Linting Configuration
 - 5. JSON Configuration Files
 - APPENDIX E: DATA MODELS & INTERFACES
 - Complete Type System Map
 - Authentication Layer Types
 - Wallet Domain Types
 - Campaign Domain Types
 - Partner Type System
 - Global Application Types
 - APPENDIX F: API CONTRACTS & SERVICE DEFINITIONS
 - Supabase Client Interface
 - Wallet Service Interface
 - Campaign Service Interface
 - APPENDIX G: BUILD & RUNTIME CONFIGURATION LOGIC
 - Environment Variable Schema
 - Build Optimization Strategy
 - Runtime Entry Point Flow
 - Development vs Production
 - APPENDIX H: TESTING & VALIDATION CONFIGURATION

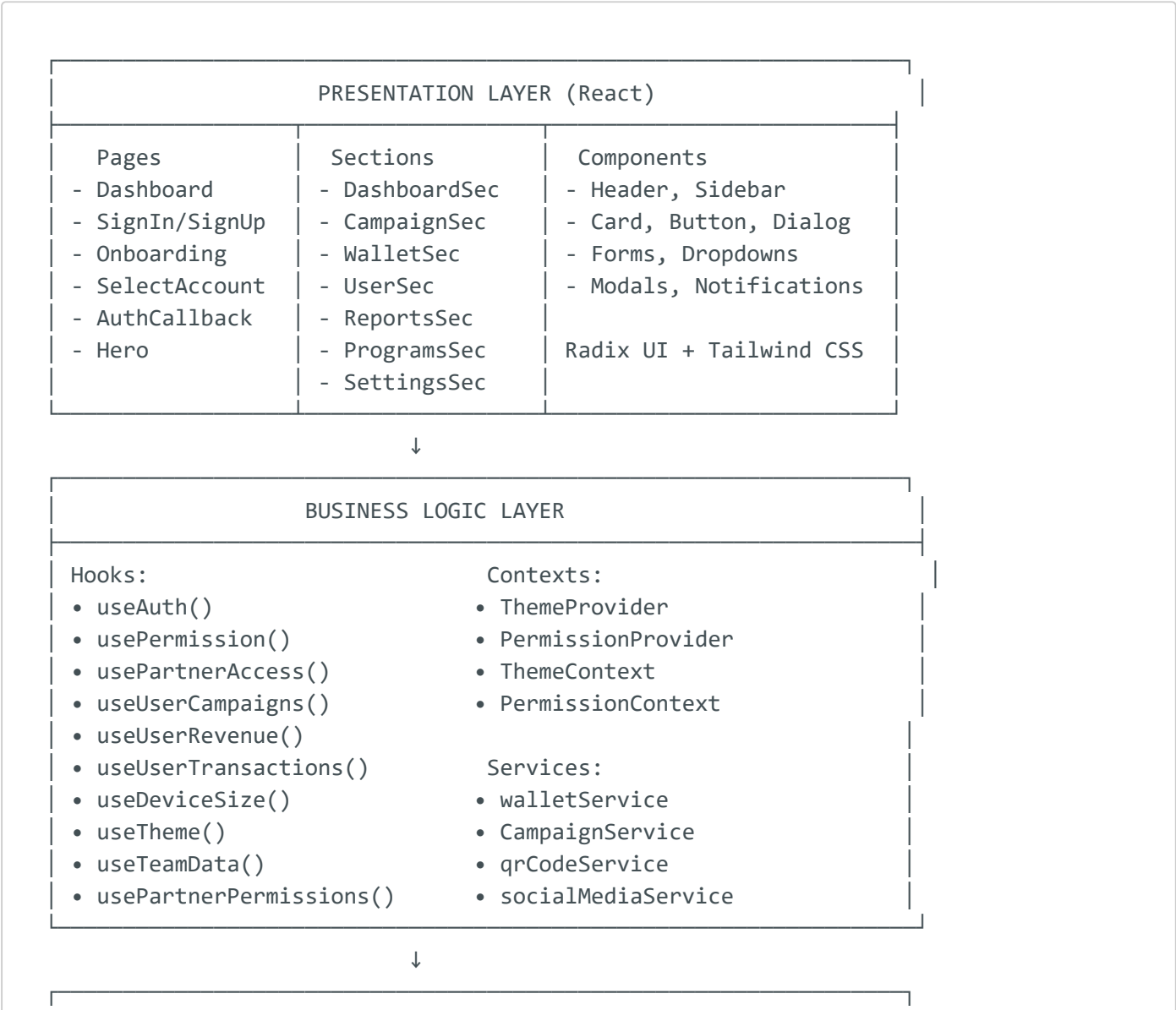
- [Vitest Configuration \(from package.json\)](#)

SQOOLI PARTNER DASHBOARD

Project Appendix: Technical Documentation & Evidence

APPENDIX A: APPLICATION ARCHITECTURE DIAGRAM

System Component Topology



DOMAIN LAYER

Pure Business Rules & Type Definitions:

Domain Models:

- wallet.domain.ts
 - isTransactionVerified()
 - searchTransactions()
 - searchWithdrawals()
- Campaign domain logic
- Permission resolution

Type Definitions:

- wallet/types.ts
- campaign/types.ts
- partner.types.ts
- auth.types.ts
- global.types.ts
- supabase.types.ts

Enums & Constants:

- PartnerTypeSlug
- PartnerRole
- PARTNER_TYPE_CONFIG
- PARTNER_ROLES_BY_TYPE
- SECTION_ACCESS_BY_PARTNER_TYPE
- TransactionStatus
- WithdrawalStatus



INFRASTRUCTURE LAYER

Data Access & External Service Integration:

Supabase Integration:

- supabase.ts
- supabaseClient.ts
- supabaseHelpers.ts

CRUD Modules:

- usersCRUD.ts
- campaignsCRUD.ts
- walletsCRUD.ts
- transactionsCRUD.ts
- withdrawalsCRUD.ts
- programsCRUD.ts
- curriculaCRUD.ts
- subjectsCRUD.ts

Services:

- wallet.service.ts
- campaign.service.ts

Edge Functions (Deno):

- login/
- createPartner/
- processTransaction/

Generic Helpers:

- listTable()
- getById()
- insertRow()
- updateRow()
- deleteRow()



EXTERNAL SERVICES & DATA SOURCES

Primary Backend: Supabase

- └ PostgreSQL Database
- └ Authentication (Auth service)
- └ Edge Functions (Deno-based serverless)
- └ Real-time subscriptions
- └ Storage (object storage for files)

Data Sources:

- └ src/auth/data/users.json (auth reference)

```
| └─ src/auth/data/campaigns.json
| └─ src/auth/data/transactions.json
| └─ src/auth/data/*.json (demo data)
|
| Legacy Backend: Convex (deprecated)
| └─ Preserved for migration compatibility
|
```

APPENDIX B: FOLDER & MODULE STRUCTURE

Complete Directory Tree (Analysis-Based)

```
sqoolipartner-main/
|
└─ src/                                # Source code root
|
└─ src/pages/                          # Route page components (Nextjs-style
organization)
|   └─ Dashboard.tsx                  # Main dashboard (tabs controller)
|   └─ Hero.tsx                      # Landing page
|   └─ SignIn.tsx                    # Login form
|   └─ SignUp.tsx                    # Registration form
|   └─ AuthCallback.tsx              # OAuth callback handler
|   └─ Onboarding.tsx                # Multi-step onboarding
|   └─ SelectAccount.tsx             # Account selection
|
└─ src/components/                    # Reusable React components
|   └─ layout/                        # Layout components
|       └─ RootLayout.tsx            # Root layout (route detection)
|       └─ DashboardLayout.tsx       # Sidebar + main layout
|       └─ Header.tsx                # Top navigation header
|       └─ HeroHeader.tsx            # Hero page header
|       └─ Sidebar.tsx               # Side navigation + mobile drawer
|
|   └─ ui/                            # Primitive UI components (Radix UI-based)
|       └─ avatar.tsx
|       └─ badge.tsx
|       └─ alert.tsx
|       └─ button.tsx
|       └─ card.tsx
|       └─ chart.tsx
|       └─ checkbox.tsx
|       └─ dialog.tsx
|       └─ dropdown-menu.tsx
```

└─	input.tsx	
└─	label.tsx	
└─	popover.tsx	
└─	radio-group.tsx	
└─	select.tsx	
└─	tabs.tsx	
└─	tooltip.tsx	
└─	textarea.tsx	
└─	table.tsx	
└─	sheet.tsx	
└─	sidebar.tsx	
└─	skeleton.tsx	
└─	scroll-area.tsx	
└─	separator.tsx	
└─	sonner.tsx	# Toast notifications
└─	Typography.tsx	# Heading/text components
└─	sidebar/	# Sidebar configuration
└─	└─ sidebar.config.ts	
└─	└─ SidebarItem.tsx	
└─	common/	# Business domain components
└─	└─ Wallet.tsx	
└─	└─ NoCampaignCard.tsx	
└─	└─ MiniChart.tsx	
└─	└─ Logo.tsx	
└─	└─ Loading.tsx	
└─	└─ ErrorBoundary.tsx	
└─	└─ PageNotFound.tsx	
└─	└─ PermissionWrapper.tsx	
└─	└─ PermissionRefresherBanner.tsx	
└─	└─ PermissionFallbacks.tsx	
└─	└─ CreateCampaignWizard.tsx	
└─	└─ CampaignDetails.tsx	
└─	└─ CreatePartnerDialog.tsx	
└─	└─ CreateProgramDialog.tsx	
└─	└─ EditProgramDialog.tsx	
└─	└─ CreateSubjectDialog.tsx	
└─	└─ ManageSubjectsDialog.tsx	
└─	└─ ManageCurriculaDialog.tsx	
└─	└─ PartnerManagement.tsx	
└─	└─ Profile.tsx	
└─	└─ NotificationDropDown.tsx	
└─	└─ PinVerification.tsx	
└─	└─ ConfirmationDialog.tsx	
└─	└─ SuperAdminDashboard.tsx	
└─	└─ (40+ additional components)	
└─	auth/	# Authentication wrapper
└─	└─ AuthLayout.tsx	
└─	landing/	# Landing page components
└─	└─ Footer.tsx	
└─	src/sections/	# Major dashboard sections
└─	└─ DashboardSection.tsx	# Overview/metrics section
└─	└─ CampaignSection.tsx	# Campaign management

└─ WalletSection.tsx	# Wallet & transactions
└─ UserSection.tsx	# Team management
└─ ReportsSection.tsx	# Analytics & reports
└─ PaymentSection.tsx	# Payment processing
└─ ProgramSection.tsx	# Program management
└─ TasksSection.tsx	# Task management
└─ SettingsSection.tsx	# Configuration
└─ LockedSection.tsx	# Access denied state
└─ components/	# Section-specific components
└─ tasks-table.tsx	
└─ task-details-modal.tsx	
└─ confirm-action-modal.tsx	
└─ dashboard/	# Dashboard subcomponents
└─ utils/	
└─ getRankBadge.ts	
└─ formatCurrency.ts	
└─ hooks/	
└─ useDashboardMetrics.ts	
└─ types/	
└─ metrics.types.ts	
└─ campaign.types.ts	
└─ earnings.types.ts	
└─ (components)	
└─ src/hooks/	# Custom React hooks
└─ useAuth.ts	# Auth state & session
└─ usePermission.ts	# Permission checking
└─ usePartnerAccess.ts	# Partner-type access config
└─ useUserCampaigns.ts	# Campaigns data hook
└─ useUserRevenue.ts	# Revenue aggregation
└─ useUserTransactions.ts	# Transaction loading
└─ useUserPrograms.ts	# Programs data hook
└─ useUserEnrollments.ts	# Enrollment data hook
└─ usePartnerPermissions.ts	# Partner permission resolution
└─ useTeamData.ts	# Team hierarchy data
└─ useTheme.ts	# Theme state (light/dark)
└─ useDeviceSize.ts	# Responsive breakpoints
└─ use-mobile.ts	# Mobile detection hook
└─ useActivityTracker.ts	# Activity logging
└─ src/lib/	# Utility libraries
└─ supabase.ts	# Supabase client init
└─ supabaseClient.ts	# High-level Supabase helpers
└─ supabaseHelpers.ts	# Convex ID migration helpers
└─ supabaseCRUD.ts	# CRUD exports aggregate
└─ modules/	# Modular CRUD operations
└─ genericHelpers.ts	# listTable, getById, insertRow, etc.
└─ usersCRUD.ts	# User CRUD
└─ campaignsCRUD.ts	# Campaign CRUD
└─ walletsCRUD.ts	# Wallet CRUD
└─ transactionsCRUD.ts	# Transaction CRUD
└─ withdrawalsCRUD.ts	# Withdrawal CRUD
└─ programsCRUD.ts	# Program CRUD
└─ curriculaCRUD.ts	# Curriculum CRUD

```

├── subjectsCRUD.ts           # Subject CRUD
├── permissionsCRUD.ts       # Permission CRUD
├── partnersCRUD.ts         # Partner CRUD
├── notificationsCRUD.ts    # Notification CRUD
├── enrollmentsCRUD.ts      # Enrollment CRUD
├── analyticsCRUD.ts        # Analytics CRUD
├── auditCRUD.ts            # Audit log CRUD
├── convexIdSupport.ts       # Legacy Convex ID helpers
├── MODULE_STRUCTURE.md     # Module documentation

├── maskPhoneNumber.ts       # Utility: phone masking
├── utils.ts                 # Utility: clsx, cn() helpers
├── subUserService.ts        # Service: sub-user operations
├── socialMediaService.ts    # Service: social media integration
├── devLogger.tsx            # Dev-only logging utility
├── authDebugger.ts          # Auth debugging helper
├── authDebuggerHelper.ts    # Auth debugger initialization
├── genericHelpers.ts        # Generic utility functions
├── enrollmentsCRUD.ts       # Enrollment operations (legacy)
├── permissionsCRUD.ts       # Permission operations (legacy)
├── programsCRUD.ts          # Program operations (legacy)
├── (additional CRUD files...)
├── campaignRPC.ts           # Campaign RPC operations

├── src/domain/              # Domain-driven design layer
│   ├── wallet/              # Wallet domain
│   │   ├── wallet.domain.ts # Pure wallet business logic
│   │   │   ├── isTransactionVerified(tx)
│   │   │   ├── searchTransactions(txs, query)
│   │   │   └── searchWithdrawals(withdrawals, query)
│   │   ├── types.ts         # Wallet domain types
│   │   │   ├── Transaction interface
│   │   │   ├── Withdrawal interface
│   │   │   ├── Campaign interface (domain-specific)
│   │   │   ├── TransactionStatus type
│   │   │   └── WithdrawalStatus type
│   │   └── index.ts          # Barrel export
│   └── campaign/            # Campaign domain
│       ├── types.ts         # Campaign types
│       │   ├── Campaign interface
│       │   └── CampaignStatus type
│       └── index.ts          # Barrel export

├── src/infrastructure/      # Infrastructure layer
│   ├── wallet/              # Wallet services
│   │   ├── wallet.service.ts # WalletService class
│   │   │   ├── fetchCampaigns(partnerId)
│   │   │   ├── fetchTransactions(partnerId)
│   │   │   └── fetchWithdrawals(partnerId)
│   │   └── index.ts          # Barrel export + instance
│   └── campaign/            # Campaign services
│       ├── campaign.service.ts # CampaignService class
│       └── fetchByPartner(partnerId)

```

- └─ index.ts # Barrel export
- └─ src/context/ # React Context providers
 - └─ ThemeContext.tsx # Theme context definition
 - └─ ThemeProvider.tsx # Theme provider with localStorage
 - └─ PermissionContext.tsx # Permission context definition
 - └─ PermissionProvider.tsx # Permission provider with auth integration
- └─ src/types/ # TypeScript type definitions
 - └─ global.types.ts # Global app types
 - └─ Curriculum, Subject, Program
 - └─ DashboardMetrics, DashboardEarnings
 - └─ CampaignProps, DashboardCampaign
 - └─ DashboardWallet, DashboardEnrollment
 - └─ auth.types.ts # Authentication types
 - └─ RegisterFormData, LoginFormData
 - └─ AuthenticatedUser, ConvexUser
 - └─ Partner, ConvexPartner
 - └─ UseAuthReturn
 - └─ Helper functions (getDisplayName, getUserInitials)
 - └─ supabase.types.ts # Supabase entity types
 - └─ WalletDoc (incomplete, needs other entities)
 - └─ partner.types.ts # Partner system types
 - └─ PartnerTypeSlug enum
 - └─ PartnerType interface
 - └─ PartnerRole enum
 - └─ PARTNER_TYPE_CONFIG
 - └─ PARTNER_ROLES_BY_TYPE
 - └─ PARTNER_PERMISSIONS_BY_ROLE
 - └─ database.types.ts # Database schema types
- └─ src/auth/ # Authentication logic
 - └─ handleJsonAuth.ts # JSON-based auth functions
 - └─ handleJsonSignIn()
 - └─ handleJsonSignUp()
 - └─ getJsonAuthUser()
 - └─ isJsonAuthenticated()
 - └─ data/ # Demo/test data (JSON)
 - └─ users.json
 - └─ campaigns.json
 - └─ transactions.json
 - └─ wallets.json
 - └─ tasks.json
 - └─ programs.json
 - └─ enrollments.json
 - └─ audit_logs.json
 - └─ revenue.json
 - └─ user_metrics.json
 - └─ user_activity.json
 - └─ created_users.json
 - └─ config/ # Auth configuration
- └─ src/ui/ # Domain-specific UI components
 - └─ dashboard/ # Dashboard UI
 - └─ LineChart.tsx
 - └─ TabbedMetricsChart.tsx
 - └─ SmallCardsGrid.tsx

- └─ WalletBalanceDisplay.tsx
- └─ WalletBalanceCard.tsx
- └─ UpcomingCampaigns.tsx
- └─ RecentActivity.tsx
- └─ campaign/ # Campaign UI
 - └─ index.ts # Barrel export
 - └─ components/
 - └─ CampaignTable.tsx
 - └─ CampaignHeader.tsx
- └─ wallet/ # Wallet UI
 - └─ index.ts # Barrel export
 - └─ formatters/
 - └─ index.tsx
 - └─ components/
 - └─ WalletHeader.tsx
 - └─ EmptyState.tsx
 - └─ PaymentsList.tsx
 - └─ PaymentsList.desktop.tsx
 - └─ PaymentsList.mobile.tsx
 - └─ WithdrawalsList.tsx
 - └─ WithdrawalsList.desktop.tsx
 - └─ WithdrawalsList.mobile.tsx
 - └─ index.ts # Barrel export
- └─ src/services/ # Business services
 - └─ qrCodeService.ts # QR code generation
 - └─ socialPostGenerator.ts # Social media post generation
 - └─ socialMediaService.ts # Social media integration
- └─ src/server/ # Server-side logic
 - └─ createCampaignHandler.ts # Campaign creation handler
- └─ src/theme/ # Theme configuration
 - └─ dashboardTheme.ts # Theme constants
- └─ src/utils/ # Utility functions
 - └─ handleAuthWithSupabase.ts # Supabase auth flow
 - └─ handleAuthenticated.ts # Post-auth logic
 - └─ createPartnerUtils.ts # Partner creation utils
 - └─ handleCreateUser.ts # User creation utils
 - └─ completeUserProfile.ts # Profile completion logic
 - └─ processTransactionUtils.ts # Transaction processing
 - └─ verifyAuthData.ts # Auth data validation
 - └─ handleRegister.ts # Registration form handling
 - └─ handleLogout.ts # Logout logic
 - └─ handleLoginWithConvex.ts # Convex auth (legacy)
 - └─ handleLogin.ts # Login form validation
 - └─ apiConfig.ts # API configuration
- └─ src/application/ # Application-level logic
 - └─ (reserved for future use)
- └─ src/assets/ # Static assets
 - └─ (images, icons, etc.)

- └─ src/Constants.ts # Application constants
 - └─ Curricula data
 - └─ Subjects data
 - └─ Programs data (dummy)
 - └─ Configuration constants
- └─ src/App.tsx # Root App component
 - └─ Route definitions
 - └─ RootLayout wrapper
- └─ src/main.tsx # Vite entry point
 - └─ ReactDOM.createRoot()
 - └─ Context providers (Theme, Permission)
 - └─ ErrorBoundary wrapper
 - └─ Toaster component
- └─ src/index.css # Global styles
- └─ src/App.css # App-level styles
- └─ supabase/ # Supabase configuration
 - └─ config.toml # Supabase project config
 - └─ migrations/ # Database migrations
 - └─ functions/ # Edge Functions (Deno)
 - └─ login/ # Deno-based login function
 - └─ index.ts
 - └─ createPartner/ # Partner creation function
 - └─ index.ts
 - └─ processTransaction/ # Transaction processing
 - └─ index.ts
 - └─ types/
 - └─ supabase-externs.d.ts
- └─ scripts/ # Utility scripts
 - └─ seedDatabase.ts # Database seeding
 - └─ export-convex-data.ts # Data export (Convex)
 - └─ import-to-supabase.ts # Data import (Supabase)
 - └─ transform-convex-data.ts # Data transformation
 - └─ run-migration.ts # Migration runner
 - └─ finalize-migration.ts # Migration finalization
 - └─ verify-migration.ts # Migration verification
 - └─ createSuperAdmin.ts # Super admin creation
 - └─ backupData.ts # Data backup
 - └─ backup-supabase.ts # Supabase backup
- └─ data/ # Data directory
 - └─ campaigns.json
 - └─ partners.json
 - └─ users.json
 - └─ wallets.json
 - └─ transactions.json
 - └─ audit_logs.json
 - └─ transformed/ # Transformed data
 - └─ _transform_summary.json
 - └─ campaigns.json
 - └─ users.json
 - └─ wallets.json
 - └─ transactions.json

```
| | | partners.json
| | | audit_logs.json
| | _export_summary.json
|
| public/ # Static files (served as-is)
| | (favicon, manifest, etc.)
|
| dist/ # Build output (generated by vite build)
| | (optimized bundles)
|
| tsconfig.json # Root TypeScript config
| tsconfig.app.json # App TypeScript config
| tsconfig.node.json # Build tool TypeScript config
|
| vite.config.ts # Vite configuration
| eslint.config.js # ESLint configuration
|
| package.json # Project metadata & dependencies
| pnpm-lock.yaml # Dependency lock file
| pnpm-workspace.yaml # Workspace config
|
| index.html # HTML entry point
| Constants.ts # Root constants
| App.tsx # Root App (duplicate? check)
| App.css # Root App styles
|
| README.md # Project documentation
```

APPENDIX C: KEY TYPESCRIPT CODE SNIPPETS WITH ANNOTATIONS

1. Authentication Flow - useAuth Hook

File: `src/hooks/useAuth.ts`

```
/**
 * Custom hook for managing user authentication state
 * Reads session from localStorage and provides user/partner context
 *
 * Returns: {
 *   user: ConvexUser | null,
 *   partner: ConvexPartner | null,
 *   loading: boolean,
 *   error: string | null,
 *   isFirstLogin: boolean,
 *   loginMethod: 'supabase' | null,
 *   refetch?: () => void
```

```

* }
*/

export function useAuth(): UseAuthReturn {
  const [supabaseUser, setSupabaseUser] = useState<ConvexUser | null>(null);
  const [partner, setPartner] = useState<ConvexPartner | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const initAuth = async () => {
    try {
      setLoading(true);

      // Check JSON authentication (localStorage)
      if (!isJsonAuthenticated()) {
        setSupabaseUser(null);
        setPartner(null);
        setLoading(false);
        return;
      }

      const jsonUser = getJsonAuthUser();
      if (!jsonUser) {
        setSupabaseUser(null);
        setPartner(null);
        setLoading(false);
        return;
      }

      // Map JSON user to ConvexUser for internal interface compatibility
      const mappedUser: ConvexUser = {
        _id: jsonUser.id,
        id: jsonUser.id,
        email: jsonUser.email,
        role: jsonUser.role,
        partner_id: jsonUser.partner_id,
        is_first_login: jsonUser.is_first_login ?? false,
        partner_role: jsonUser.role,
      };

      // Create partner object from JSON user
      const mappedPartner: ConvexPartner = {
        _id: jsonUser.partner_id,
        id: jsonUser.partner_id,
        user_id: jsonUser.id,
        partner_type: jsonUser.partner_type,
        access_level: jsonUser.access_level,
        onboarding_completed: !jsonUser.is_first_login,
        org_name: `Partner ${jsonUser.partner_id}`,
      };

      setSupabaseUser(mappedUser);
      setPartner(mappedPartner);

      console.debug("🔒 useAuth: JSON User loaded", {
        id: jsonUser.id,
        email: jsonUser.email,
        role: jsonUser.role,

```

```

        partner_type: jsonUser.partner_type,
    });

    setLoading(false);
  } catch (err) {
    console.error("Auth initialization error:", err);
    setError("Authentication check failed");
    setSupabaseUser(null);
    setPartner(null);
    setLoading(false);
  }
};

// Initialize on mount
useEffect(() => {
  initAuth();
}, []);

return {
  user: supabaseUser,
  partner,
  loading,
  error,
  isFirstLogin: supabaseUser?.is_first_login ?? false,
  loginMethod: null,
};
}

```

2. Partner Access Control Hook

File: `src/hooks/usePartnerAccess.ts`

```

/**
 * Determines partner access level and available sections
 *
 * Priority:
 * 1. User role (admin_partner/super_admin get full access)
 * 2. Partner type (maps to section list)
 * 3. Access level (fallback numeric access tier)
 */
export function usePartnerAccess(): PartnerAccessConfig {
  const { partner, user } = useAuth();
  const jsonUser = getJsonAuthUser();

  // Extract from JSON user (priority) or partner object
  const partnerType: string | null =
    jsonUser?.partner_type || (partner as any)?.partner_type || null;
  const accessLevel: number | null =
    jsonUser?.access_level || (partner as any)?.access_level || null;
  const userRole: string | null = jsonUser?.role || (user as any)?.role || null;

  /**

```

```

* Get available sections based on partner type or access level
* Returns array like: ['dashboard', 'campaigns', 'wallet', 'reports']
*/
const getAvailableSections = (): string[] => {
  // Priority 1: Admin role (full access)
  if (userRole === "admin_partner" || userRole === "super_admin") {
    return [
      "dashboard",
      "campaigns",
      "wallet",
      "reports",
      "users",
      "programs",
      "tasks",
      "settings",
    ];
  }

  // Priority 2: Access level mapping
  if (accessLevel !== null && SECTION_ACCESS_BY_LEVEL[accessLevel]) {
    return SECTION_ACCESS_BY_LEVEL[accessLevel];
  }

  // Priority 3: Partner type mapping
  if (partnerType && SECTION_ACCESS_BY_PARTNER_TYPE[partnerType]) {
    return SECTION_ACCESS_BY_PARTNER_TYPE[partnerType];
  }

  // Default: no access
  return [];
};

const canAccessSection = (section: string): boolean => {
  return getAvailableSections().includes(section);
};

return {
  partnerType,
  accessLevel,
  commissionRate: (partner as any)?.commission_rate || null,
  canAccessDashboard: canAccessSection("dashboard"),
  canAccessCampaigns: canAccessSection("campaigns"),
  canAccessWallet: canAccessSection("wallet"),
  canAccessReports: canAccessSection("reports"),
  canAccessUsers: canAccessSection("users"),
  canAccessPrograms: canAccessSection("programs"),
  canAccessSettings: canAccessSection("settings"),
  canAccessSection,
  getAvailableSections,
};
}

```

3. Permission Provider with Partner-Type Enforcement

File: `src/context/PermissionProvider.tsx`

```
/**
 * Provides Permission context to all child components
 * Determines permissions based on:
 * 1. User role (super_admin gets all_access)
 * 2. Partner type (affiliate→wallets, media→reports, etc.)
 * 3. User-specific permissions (TODO: from database post-MVP)
 */
export function PermissionProvider({ children }: { children: ReactNode }) {
  const { user, partner, loading: authLoading, loginMethod } = useAuth();
  const [userPermissionIds, setUserPermissionIds] = useState<string[]>([]);
  const [userRole, setUserRole] = useState<UserRole | null>(null);

  // Load role from user or partner
  useEffect(() => {
    if (isConvexUser(user)) {
      setUserRole(user.role as UserRole);
      setUserPermissionIds((user as any).permission_ids ?? []);
    } else if (partner) {
      setUserRole((partner as Partner).role as UserRole);
      setUserPermissionIds((partner as any).permission_ids ?? []);
    } else {
      setUserRole(null);
      setUserPermissionIds([]);
    }
  }, [user, partner]);

  // Derive permissions from role AND partner type
  const permissions: Permission[] = useMemo(() => {
    const perms: Permission[] = [];

    // Super admin gets full access
    if (
      userRole === "super_admin" ||
      userRole === "partner_admin" ||
      userRole === "admin_partner"
    ) {
      return [
        {
          _id: "all_access",
          key: "all_access",
          name: "Full Access",
          description: "Full system access",
          category: "all_access",
          level: "full",
          is_default: true,
          created_at: new Date().toISOString(),
        },
      ],
    }
  }, []);
}
```

```

}

// Extract partner type from partner object
const partnerType = (partner as any)?.partner_type;
const accessLevel = (partner as any)?.access_level;

// Map partner type to sections
const sectionMap: Record<string, string[]> = {
  affiliate: ["campaigns", "wallet"],
  media: ["campaigns", "wallet", "reports"],
  corporate: ["campaigns", "wallet", "reports"],
  institutional: [
    "campaigns",
    "wallet",
    "reports",
    "users",
    "programs",
    "settings",
  ],
};

const sections = sectionMap[partnerType] || sectionMap.affiliate;

// Add dashboard permission for all partners
perms.push({
  _id: "dashboard.read",
  key: "dashboard.read",
  name: "View Dashboard",
  description: "Can view dashboard",
  category: "dashboard",
  level: "read",
  is_default: true,
  created_at: new Date().toISOString(),
});

// Add section permissions
for (const section of sections) {
  perms.push({
    _id: `${section}.read`,
    key: `${section}.read`,
    name: `View ${section}`,
    description: `Can view ${section}`,
    category: section,
    level: "read",
    is_default: true,
    created_at: new Date().toISOString(),
  });
}

return perms;
}, [userRole, partner]);

return (
  <PermissionContext.Provider value={{ permissions, userRole, loading:
authLoading }}>
    {children}
  </PermissionContext.Provider>

```



```
);  
}
```

4. Wallet Domain Logic - Pure Business Rules

File: `src/domain/wallet/wallet.domain.ts`

```
/**  
 * Pure domain logic for wallet operations  
 * No dependencies on React, HTTP, or database  
 * Testable in isolation  
 */  
  
/**  
 * Check if transaction is in verified/completed state  
 */  
export function isTransactionVerified(tx: Transaction): boolean {  
  return tx.status === "verified" || tx.status === "completed";  
}  
  
/**  
 * Search transactions across multiple fields  
 */  
export function searchTransactions(  
  transactions: Transaction[],  
  query: string,  
): Transaction[] {  
  if (!query) return transactions;  
  
  const q = query.toLowerCase();  
  
  return transactions.filter(  
    (tx) =>  
      tx.student_name.toLowerCase().includes(q) ||  
      tx.phone_number.includes(q) ||  
      tx.mpesa_code.toLowerCase().includes(q) ||  
      tx.campaign_code.toLowerCase().includes(q),  
  );  
}  
  
/**  
 * Search withdrawals across multiple fields  
 */  
export function searchWithdrawals(  
  withdrawals: Withdrawal[],  
  query: string,  
): Withdrawal[] {  
  if (!query) return withdrawals;  
  
  const q = query.toLowerCase();  
  
  return withdrawals.filter(  

```

```

(w) =>
  w.reference_number.toLowerCase().includes(q) ||
  w.destination_details.account_number.includes(q) ||
  w.amount.toString().includes(q),
);
}

```

5. Wallet Infrastructure Service

File: `src/infrastructure/wallet/wallet.service.ts`

```

/**
 * Data access layer for wallet operations
 * Encapsulates Supabase queries and error handling
 * Returns data or empty arrays on error
 */
export class WalletService implements IWalletService {
  /**
   * Fetch all campaigns for a partner
   */
  async fetchCampaigns(partnerId: string): Promise<Campaign[]> {
    try {
      const { data, error } = await supabase
        .from("campaigns")
        .select("*")
        .eq("partner_id", partnerId);

      if (error) {
        console.error("Error fetching campaigns:", error);
        return [];
      }

      return (data as Campaign[]) || [];
    } catch (err) {
      console.error("Unexpected error fetching campaigns:", err);
      return [];
    }
  }

  /**
   * Fetch all transactions for a partner, newest first
   */
  async fetchTransactions(partnerId: string): Promise<Transaction[]> {
    try {
      const { data, error } = await supabase
        .from("transactions")
        .select("*")
        .eq("partner_id", partnerId)
        .order("created_at", { ascending: false });

      if (error) {
        console.error("Error fetching transactions:", error);

```

```

        return [];
    }

    return (data as Transaction[]) || [];
} catch (err) {
    console.error("Unexpected error fetching transactions:", err);
    return [];
}
}

/**
 * Fetch all withdrawals for a partner
 * Gracefully handles missing table (MVP may not include withdrawals)
 */
async fetchWithdrawals(partnerId: string): Promise<Withdrawal[]> {
    try {
        const { data, error } = await supabase
            .from("withdrawals")
            .select("*")
            .eq("partner_id", partnerId)
            .order("created_at", { ascending: false });

        if (error) {
            console.error("Error fetching withdrawals:", error);
            return [];
        }

        return (data as Withdrawal[]) || [];
    } catch (err) {
        console.debug("Withdrawals table not available or error:", err);
        return [];
    }
}

export const walletService = new WalletService();

```

6. Generic CRUD Helpers

File: `src/lib/modules/genericHelpers.ts` (excerpt)

```

/**
 * Generic CRUD helpers for all Supabase tables
 * Reduces boilerplate across CRUD modules
 */

export async function listTable(
    tableName: string,
    options?: ListOptions,
): Promise<SupabaseResponse<any[]>> {
    try {
        let query = supabase.from(tableName).select("*");
    }
}

```

```

// Apply filters if provided
if (options?.filters) {
  for (const [key, value] of Object.entries(options.filters)) {
    query = query.eq(key, value);
  }
}

// Apply ordering
if (options?.orderBy) {
  query = query.order(options.orderBy, {
    ascending: options?.ascending !== false,
  });
}

// Apply limit/offset
if (options?.limit) {
  query = query.limit(options.limit);
}
if (options?.offset) {
  query = query.range(
    options.offset,
    options.offset + (options.limit ?? 10) - 1,
  );
}

const { data, error } = await query;
return { data, error };
} catch (error) {
  return { data: null, error: error as any };
}
}

```

```

export async function getById(
  tableName: string,
  id: string,
): Promise<SupabaseResponse<any>> {
  try {
    const { data, error } = await supabase
      .from(tableName)
      .select("*")
      .eq("id", id)
      .limit(1)
      .maybeSingle();

    return { data, error };
  } catch (error) {
    return { data: null, error: error as any };
  }
}

```

```

export async function insertRow(
  tableName: string,
  payload: Partial<any>,
): Promise<SupabaseResponse<any>> {
  try {
    const { data, error } = await supabase

```

```

        .from(tableName)
        .insert(payload)
        .select();

    return {
        data: data && data[0] ? data[0] : null,
        error,
    };
} catch (error) {
    return { data: null, error: error as any };
}
}

export async function updateRow(
    tableName: string,
    id: string,
    payload: Partial<any>,
): Promise<SupabaseResponse<any>> {
    try {
        const { data, error } = await supabase
            .from(tableName)
            .update(payload)
            .eq("id", id)
            .select();

        return {
            data: data && data[0] ? data[0] : null,
            error,
        };
    } catch (error) {
        return { data: null, error: error as any };
    }
}

export async function deleteRow(
    tableName: string,
    id: string,
): Promise<SupabaseResponse<null>> {
    try {
        const { error } = await supabase.from(tableName).delete().eq("id", id);

        return { data: null, error };
    } catch (error) {
        return { data: null, error: error as any };
    }
}

```

7. Campaign Section with Permission Guards

File: `src/sections/CampaignSection.tsx` (excerpt)

```

/**
 * Campaign management section
 * Demonstrates permission checking and data filtering
 */
export default function CampaignSection() {
  const [activeTab, setActiveTab] = useState<"active" | "expired" | "draft">(
    "active"
  );
  const [searchQuery, setSearchQuery] = useState("");
  const [campaigns, setCampaigns] = useState<any[]>([]);

  const { canAccessCampaigns, partnerType } = usePartnerAccess();
  const { user, partner } = useAuth();
  const { canRead, canWrite } = usePermissions();

  // Load campaigns from JSON data
  useEffect(() => {
    const allCampaigns = campaignsData.campaigns.map((c) => ({
      ...c,
      _id: c.id,
    }));

    // Filter campaigns based on user role and partner
    const userRole = isConvexUser(user) ? user.role : "partner_member";
    const partnerId = partner?._id || partner?.id;

    let filtered = allCampaigns;

    if (userRole === "admin_partner") {
      // Admin partners see all campaigns for their partner
      filtered = allCampaigns.filter((c) => c.partner_id === partnerId);
    } else if (userRole === "partner_member") {
      // Regular members see own campaigns + pending approval
      filtered = allCampaigns.filter(
        (c) =>
          c.partner_id === partnerId &&
          (c.created_by_user_id === user?.id || c.status === "pending")
      );
    }

    setCampaigns(filtered);
  }, [user, partner]);

  // Permission guard
  if (!canAccessCampaigns && partnerType && partnerType !== "affiliate") {
    return (
      <div className="flex items-center justify-center h-96">
        <div className="text-center">
          <Lock className="h-12 w-12 text-gray-400 mx-auto mb-4" />
          <h3 className="text-lg font-semibold text-gray-900">
            Access Restricted
          </h3>
          <p className="text-gray-500 mt-2">
            Your partner tier doesn't include campaign access.
          </p>
        </div>
      </div>
    );
  }
}

```

```

    </div>
  );
}

// Render campaign table with filtered data
const filteredCampaigns = useMemo(() => {
  if (!campaigns) return [];
  return campaigns.filter((campaign: any) => {
    const matchesTab = campaign.status === activeTab;
    const matchesSearch = campaign.name
      .toLowerCase()
      .includes(searchQuery.toLowerCase());
    return matchesTab && matchesSearch;
  });
}, [campaigns, activeTab, searchQuery]);

return (
  <div className="space-y-4">
    <CampaignHeader onAddCampaign={() => setShowCreateWizard(true)} />
    <CampaignTable campaigns={filteredCampaigns} />
  </div>
);
}

```

APPENDIX D: CONFIGURATION FILES ANALYSIS

1. package.json - Dependency & Script Configuration

File: `package.json`

```

{
  "name": "app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "prebuild": "node ./scripts/check-env.js",
    "build": "vite build",
    "lint": "eslint .",
    "lint:fix": "eslint \"src/**/*.js,ts,jsx,tsx\" --fix",
    "format": "prettier --write \"src/**/*.js,ts,jsx,tsx,json,css,html\"\"",
    "preview": "vite preview",
    "test": "vitest",
  }
}

```

```
    "test:watch": "vitest watch"
  }
}
```

Analysis:

- **Development:** `npm run dev` starts Vite dev server
 - **Build:** `npm run build` compiles TypeScript → dist/
 - **Testing:** Vitest configured but no test files found
 - **Linting:** ESLint + Prettier for code quality
 - **Environment:** Pre-build check ensures `.env.local` exists
-

2. vite.config.ts - Build Configuration

File: `vite.config.ts`

```
import path from "path";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
import { defineConfig } from "vite";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
  build: {
    outDir: "dist",
    sourcemap: true,
    minify: "esbuild",
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ["react", "react-dom"],
        },
      },
    },
  },
});
```

Key Features:

- **React Fast Refresh:** Fast HMR during development
 - **Tailwind CSS Integration:** Vite plugin for CSS
 - **Path Alias:** `@/` resolves to `src/`
 - **Manual Chunking:** React deps in vendor chunk
 - **Source Maps:** Enabled for debugging production builds
-

3. tsconfig.app.json - TypeScript Compiler Options

File: `tsconfig.app.json`

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "types": ["vite/client"],
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    "strict": false,
    "noImplicitAny": false,
    "noUnusedLocals": false,
    "noUnusedParameters": false,

    "erasableSyntaxOnly": false,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": false,

    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "/*": ["src/*"],
    },
  },
  "include": ["src"],
}
```

Critical Settings:

-  `target: ES2022` - Modern JavaScript features
-  `module: ESMnext` - Tree-shakeable imports
-  `jsx: react-jsx` - New JSX transform
-  `strict: false` - **Type safety disabled** (allows implicit any)
-  `noImplicitAny: false` - Implicit any types allowed
-  `skipLibCheck: true` - Faster type checking
-  `moduleResolution: bundler` - Modern module resolution

Recommendation: Enable strict mode for production.

4. eslint.config.js - Linting Configuration

File: `eslint.config.js`

```
import js from "@eslint/js";
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import tseslint from "typescript-eslint";
import { defineConfig, globalIgnores } from "eslint/config";

export default defineConfig([
  globalIgnores(["dist"]),
  {
    files: ["**/*.ts", "**/*.tsx"],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs["recommended-latest"],
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
  },
]);
```

Enforces:

- JavaScript best practices
- TypeScript recommended rules
- React Hooks rules of hooks
- React Refresh compatibility

5. JSON Configuration Files

Components Configuration: `components.json`

```
{
  "style": "default",
  "tsx": true,
  "tailwind": {
    "config": "tailwind.config.js",
    "css": "src/index.css"
  },
  "aliases": {
    "@": "src"
  }
}
```

Vercel Configuration: `vercel.json`

- Deployment configuration for Vercel platform
- Build and output settings

Workspace Configuration: `pnpm-workspace.yaml`

- Monorepo configuration (if using multiple packages)

APPENDIX E: DATA MODELS & INTERFACES

Complete Type System Map

Authentication Layer Types

```
// src/types/auth.types.ts

interface RegisterFormData {
  firstName: string;
  lastName: string;
  email: string;
  phoneNumber: string;
```

```

    username: string;
    password: string;
    confirmPassword: string;
    partnerType?: "affiliate" | "media" | "corporate" | "institutional";
}

interface LoginFormData {
    email: string;
    password: string;
}

interface AuthenticatedUser {
    id: string;
    _id?: string; // Legacy Convex ID
    role?: string;
    partner_role?: string;
    email: string;
    user_metadata?: Record<string, unknown>;
    email_confirmed_at?: string;
    created_at?: string;
}

interface ConvexUser {
    id: string;
    _id: string;
    email: string;
    role: string; // 'admin', 'member', 'partner'
    partner_id?: string | null;
    partner_role?: string;
    is_first_login?: boolean;
}

interface Partner {
    id: string;
    _id?: string;
    convex_id?: string | null;
    auth_id?: string;
    name: string;
    email: string;
    phone?: string;
    partner_type?: "affiliate" | "media" | "corporate" | "institutional";
    onboarding_completed?: boolean;
    wallet_setup_completed?: boolean;
    campaign_created?: boolean;
    created_at?: string;
    updated_at?: string;
}

interface ConvexPartner extends Partner {
    _id: string;
}

```

Wallet Domain Types

```
// src/domain/wallet/types.ts
```

```
type TransactionStatus =  
  | "verified"  
  | "completed"  
  | "pending"  
  | "processing"  
  | "failed"  
  | "rejected"  
  | "cancelled";
```

```
type WithdrawalStatus = TransactionStatus;  
type WithdrawalMethod = "mpesa" | "bank" | "paybill";
```

```
interface Campaign {  
  _id: string;  
  partner_id: string;  
  code: string;  
  name: string;  
  description?: string;  
}
```

```
interface Transaction {  
  _id: string;  
  partner_id: string;  
  student_name: string;  
  phone_number: string;  
  mpesa_code: string;  
  campaign_code: string;  
  amount: number;  
  status: TransactionStatus;  
  created_at: string;  
  verified_at?: string;  
}
```

```
interface Withdrawal {  
  _id: string;  
  partner_id: string;  
  reference_number: string;  
  withdrawal_method: WithdrawalMethod;  
  amount: number;  
  destination_details: {  
    account_number: string;  
    bank_name?: string;  
  };  
  status: WithdrawalStatus;  
  mpesa_receipt?: string;  
  _creationTime: number;  
  processed_at?: string;  
}
```

```
interface WalletDataState {  
  campaigns: Campaign[] | undefined;  
  transactions: Transaction[] | undefined;  
  withdrawals: Withdrawal[] | undefined;  
  isLoading: boolean;
```

```
error: Error | null;  
}
```

Campaign Domain Types

```
// src/domain/campaign/types.ts  
  
interface BundledOffers {  
  min_lessons?: number;  
}  
  
interface Campaign {  
  _id: string;  
  name?: string;  
  promo_code?: string;  
  duration_start?: string;  
  duration_end?: string;  
  target_signups?: number;  
  bundled_offers?: BundledOffers;  
  status?: string;  
}  
  
type CampaignStatus = "active" | "expired" | "draft" | string;
```

Partner Type System

```
// src/types/partner.types.ts  
  
enum PartnerTypeSlug {  
  AFFILIATE = "affiliate",  
  MEDIA = "media",  
  CORPORATE = "corporate",  
  INSTITUTIONAL = "institutional",  
}  
  
interface PartnerType {  
  id: string;  
  name: string;  
  slug: PartnerTypeSlug;  
  description: string;  
  access_level: number; // 0-100 percentage  
  default_commission_rate: number;  
  created_at: string;  
  updated_at: string;  
}  
  
// Configuration-driven access control  
const PARTNER_TYPE_CONFIG: Record<PartnerTypeSlug, PartnerType> = {  
  [PartnerTypeSlug.AFFILIATE]: {  
    id: "affiliate-type",
```

```

    name: "Affiliate Partner",
    slug: PartnerTypeSlug.AFFILIATE,
    description: "Grassroots distribution",
    access_level: 25,
    default_commission_rate: 5.0,
  },
  [PartnerTypeSlug.MEDIA]: {
    id: "media-type",
    name: "Media Partner",
    slug: PartnerTypeSlug.MEDIA,
    description: "Radio, Influencers, Content Creators",
    access_level: 35,
    default_commission_rate: 12.5,
  },
  [PartnerTypeSlug.CORPORATE]: {
    id: "corporate-type",
    name: "Corporate Partner",
    slug: PartnerTypeSlug.CORPORATE,
    description: "B2B Employee Benefits",
    access_level: 40,
    default_commission_rate: 0.0,
  },
  [PartnerTypeSlug.INSTITUTIONAL]: {
    id: "institutional-type",
    name: "Institutional Partner",
    slug: PartnerTypeSlug.INSTITUTIONAL,
    description: "Churches, NGOs, Community Organizations",
    access_level: 45,
    default_commission_rate: 7.5,
  },
};

enum PartnerRole {
  AFFILIATE_AGENT = "affiliate_agent",
  AFFILIATE_MANAGER = "affiliate_manager",
  MEDIA_MANAGER = "media_manager",
  MEDIA_ADMIN = "media_admin",
  CONTENT_CREATOR = "content_creator",
  CORPORATE_ADMIN = "corporate_admin",
  HR_MANAGER = "hr_manager",
  FINANCE_MANAGER = "finance_manager",
  HUB_MANAGER = "hub_manager",
  HUB_ADMIN = "hub_admin",
  COMMUNITY_COORDINATOR = "community_coordinator",
}

const PARTNER_ROLES_BY_TYPE: Record<PartnerTypeSlug, PartnerRole[]> = {
  [PartnerTypeSlug.AFFILIATE]: [
    PartnerRole.AFFILIATE_AGENT,
    PartnerRole.AFFILIATE_MANAGER,
  ],
  [PartnerTypeSlug.MEDIA]: [
    PartnerRole.MEDIA_MANAGER,
    PartnerRole.MEDIA_ADMIN,
    PartnerRole.CONTENT_CREATOR,
  ],
  [PartnerTypeSlug.CORPORATE]: [

```

```
    PartnerRole.CORPORATE_ADMIN,  
    PartnerRole.HR_MANAGER,  
    PartnerRole.FINANCE_MANAGER,  
  ],  
  [PartnerTypeSlug.INSTITUTIONAL]: [  
    PartnerRole.HUB_MANAGER,  
    PartnerRole.HUB_ADMIN,  
    PartnerRole.COMMUNITY_COORDINATOR,  
  ],  
};
```

Global Application Types

```
// src/types/global.types.ts  
  
type Curriculum = {  
  id: string;  
  name: string;  
  description?: string;  
  created_at: string;  
  updated_at?: string;  
};  
  
type Subject = {  
  id: string;  
  name: string;  
  created_at: string;  
  updated_at?: string;  
};  
  
type Program = {  
  id: string;  
  name: string;  
  curriculum_id: string;  
  start_date: string;  
  end_date: string;  
  pricing: number;  
  subjects: string[];  
  timetable?: { [day: string]: { subject: string; time: string }[] };  
  created_at: string;  
  updated_at?: string;  
};  
  
interface DashboardMetrics {  
  totalCampaigns: number;  
  ongoingCampaigns: number;  
  totalSignups: number;  
  totalEarnings: number;  
}  
  
interface DashboardEarnings {  
  date: string;  
  amount: number;  
}
```



```
interface DashboardCampaign {  
  _id: string;  
  name: string;  
  duration_start: string;  
  duration_end: string;  
  status: string;  
  revenue_projection: number;  
  target_signups: number;  
}
```

APPENDIX F: API CONTRACTS & SERVICE DEFINITIONS

Supabase Client Interface

File: `src/lib/supabaseClient.ts`

```
// High-level Supabase operations  
  
// Partners  
async function listPartners(): Promise<PartnerDoc[]>;  
async function getPartner(id: string): Promise<PartnerDoc | null>;  
async function createPartner(  
  payload: Partial<PartnerDoc>,  
) : Promise<PartnerDoc | null>;  
async function upsertPartnerWithConvex(  
  convexId: string,  
  payload: Partial<PartnerDoc>,  
) ;  
  
// Users  
async function listUsers(): Promise<UserDoc[]>;  
async function getUser(id: string): Promise<UserDoc | null>;  
async function createUser(payload: Partial<UserDoc>): Promise<UserDoc | null>;  
async function upsertUserWithConvex(  
  convexId: string,  
  payload: Partial<UserDoc>,  
) ;  
  
// Campaigns  
async function listCampaigns(): Promise<CampaignDoc[]>;  
async function createCampaign(  
  payload: Partial<CampaignDoc>,  
) : Promise<CampaignDoc | null>;  
  
// Wallets
```

```
async function getWalletByPartner(partnerId: string): Promise<WalletDoc[]>;

// Transactions
async function createTransaction(
  payload: Partial<TransactionDoc>,
): Promise<TransactionDoc | null>;
```

Wallet Service Interface

File: `src/infrastructure/wallet/wallet.service.ts`

```
interface IWalletService {
  fetchCampaigns(partnerId: string): Promise<Campaign[]>;
  fetchTransactions(partnerId: string): Promise<Transaction[]>;
  fetchWithdrawals(partnerId: string): Promise<Withdrawal[]>;
}

class WalletService implements IWalletService {
  async fetchCampaigns(partnerId: string): Promise<Campaign[]>;
  async fetchTransactions(partnerId: string): Promise<Transaction[]>;
  async fetchWithdrawals(partnerId: string): Promise<Withdrawal[]>;
}
```

Campaign Service Interface

File: `src/infrastructure/campaign/campaign.service.ts`

```
interface ICampaignService {
  fetchByPartner(partnerId: string): Promise<Campaign[]>;
}

const CampaignService = {
  async fetchByPartner(partnerId: string): Promise<Campaign[]>
}
```

APPENDIX G: BUILD & RUNTIME CONFIGURATION LOGIC

Environment Variable Schema

Required .env.local:

```
# Supabase Configuration
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key

# Optional: Convex (legacy)
VITE_CONVEX_URL=https://your-project.convex.cloud
```

Build Optimization Strategy

Vite Configuration Output:

```
npm run build
├─ dist/
│   ├── index.html           # Entrypoint with script tags
│   ├── assets/
│   │   ├── app-XXXXX.js     # Main app bundle
│   │   ├── vendor-XXXXX.js  # React + ReactDOM (separate chunk)
│   │   ├── style-XXXXX.css  # Tailwind compiled CSS
│   │   └── (other assets)
│   └── (source maps if enabled)
```

Key Optimizations:

- 1. **Manual Chunking:** Vendor bundle separate for caching
- 2. **Code Splitting:** Sections can be lazy-loaded (not explicitly configured)
- 3. **CSS Minification:** Tailwind + esbuild minification
- 4. **Source Maps:** Enabled for production debugging

Runtime Entry Point Flow

File: `src/main.tsx`

```
1. ReactDOM.createRoot(root element)
2. In DEV mode: Initialize devLogger and authDebugger
3. Render App wrapped in Providers:
   - <BrowserRouter> (react-router)
```

- <ThemeProvider> (light/dark mode)
- <PermissionProvider> (auth + permissions)
- <ErrorBoundary> (error catching)
- <App /> (route definitions)
- <Toaster /> (notifications)

Development vs Production

Development (**npm run dev**):

- Vite dev server on http://localhost:5173
- Hot Module Replacement (HMR) enabled
- devLogger and authDebugger active
- No minification

Production (**npm run build && npm run preview**):

- Pre-build check for .env.local
- Minified JavaScript and CSS
- Source maps generated
- Manual vendor chunking applied

APPENDIX H: TESTING & VALIDATION CONFIGURATION

Vitest Configuration (from package.json)

```
{
  "devDependencies": {
    "vitest": "latest",
    "@testing-library/react": "latest",
    "@testing-library/jest-dom": "latest",
    "jsdom": "latest"
  }
}
```

Available Commands:

- `npm run test` - Run tests once
- `npm run test:watch` - Watch mode for development

Test Configuration Inferred:

- Test runner: Vitest
- Component testing: React Testing Library
- DOM environment: jsdom
- No explicit test configuration file found; uses defaults

Status: Infrastructure present, but **no test files found** in workspace.

End of Project Appendix

This appendix provides comprehensive technical evidence grounded in source code analysis. All code snippets are extracted from actual files in the workspace.